

Online Motion Generation via Tangential Sampling-Based MPC Around Nonconvex Obstacles

Guangbao Zhao[✉], Ninglong Jin, Jianhua Wu[✉], *Member, IEEE*, and Zhenhua Xiong[✉], *Member, IEEE*

Abstract—Collision-free motion planning has a well-established research history, but the majority of studies have been centered around Euclidean space and conducted offline. The primary challenge in online motion planning lies in circumventing local minima, which become more pronounced in configuration space. We propose an online method for generating collision-free motion in configuration space that effectively avoids local minima. Our approach decomposes the optimal velocity into nominal and tangential components, with the tangential velocity optimized to facilitate escape from local minima. The tangential velocity is defined in the tangential space, with its normal direction determined by the gradient of the nearest distance between the robot and obstacles, relative to the robot's states. Direct optimization of the tangential velocity is challenging due to its dependence on varying tangent spaces. To address this, we represent the tangential velocity using the orthogonal basis of the tangent space, decoupling it from the varying tangent space. This allows explicit optimization of the tangential velocity by optimizing its components. Additionally, we introduce a warm-start operator in the tangent space to ensure the consistency and convergence. Furthermore, we propose a dynamic weight based on proximity to local minima to balance the tangential and nominal velocities, forming the optimal velocity. The effectiveness of our approach in avoiding local minima is validated through simulations and physical experiments.

Index Terms—Nonconvex obstacle avoidance, sampling-based MPC, tangential configuration space optimization.

I. INTRODUCTION

MODERN robotic systems usually need to perform complex tasks in the presence of obstacles. While collision-free motion planning has a well-established research history, most studies focus on Euclidean space and are conducted offline. Online motion generation approaches can suffer from local minima induced by non-convex obstacles, which become more pronounced in high-dimensional configuration space.

Received 26 November 2024; accepted 26 March 2025. Date of publication 14 April 2025; date of current version 25 April 2025. This article was recommended for publication by Associate Editor S. Zhao and Editor A. Bera upon evaluation of the reviewers' comments. This work was supported in part by Shanghai Technology Innovation Action Plan under Grant 24511103303, in part by Jiangsu Provincial Science and Technology Department Program Special Funds (Basic Research on Frontier Leading Technologies) under Project SBK2023050162, and in part by the State Key Laboratory of Mechanical System and Vibration under Grant MSVZD202205. (Corresponding author: Jianhua Wu.)

The authors are with the State Key Laboratory of Mechanical System, and Vibration, Institute of Robotics, School of Mechanical Engineering, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: zhaoguangbao@sjtu.edu.cn; ninglong_jin@sjtu.edu.cn; wujh@sjtu.edu.cn; mexiong@sjtu.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/LRA.2025.3560885>, provided by the authors.

Digital Object Identifier 10.1109/LRA.2025.3560885

Classical online collision-free motion generation methods include the Artificial Potential Fields (APF) [1] and the local Modulation Dynamical System (ModDS). The APF method is generally limited to convex obstacles due to its susceptibility to local minima. The convergence of ModDS can only be guaranteed for conservative obstacle types, such as convex obstacles [2] or star-shaped ones [3]. Further, saddle-point local minima remain for such shapes. These limitations reduce the effectiveness of APF and ModDS in environments with complex-shaped obstacles. Lukas et al. [4] proposed an improvement upon the ModDS by introducing additional tangential velocities through an additive nonlinear function based on reference points to alleviate spurious attractors. However, determining these reference points for high-dimensional concave obstacles remains challenging. Koptev et al. [5] proposed an approach to generate locally active obstacle-tangent deflections in potential local minima presented in ModDS. In addition to potential-based methods, optimization-based approaches such as [6] reformulated reactive motion planning as a quadratic programming problem, but struggle to escape local minima due to their single-horizon formulation. Recently, Model-Predictive Path Integral Control (MPPI) [7] has been used to generate collision-free trajectories online [8], [9], but these methods still face challenges in handling local minima.

The tangential velocity is essential for steering the robot out of local minima. It is defined in the tangential space, where its normal direction is determined by the gradient of the nearest distance between the robot and obstacles, relative to the robot's states. Determining an appropriate velocity online, especially the optimal tangential velocity to escape local minima, is challenging. The primary difficulty lies in maintaining consistency in the tangential velocity, ensuring the direction remains stable and aligned with the target. To address this, we decompose the optimal velocity into tangential and nominal components rather than directly optimizing the overall velocity. We explicitly optimize the components along the tangent space basis to compute the tangential velocity. We introduce a warm start operator in the tangent space and leverage the receding horizon of the MPPI algorithm to ensure the convergence and consistency in tangential velocity optimization. We also adopt a dynamic weight based on the distance to local minima to combine the tangential and nominal velocities. The nominal velocity can be derived from a linear dynamical system or ModDS. The architecture of the optimization process is illustrated in Fig. 1. We refer to the proposed approach as *MPPI-Tan*. Simulation and physical experiments demonstrate its effectiveness in navigating

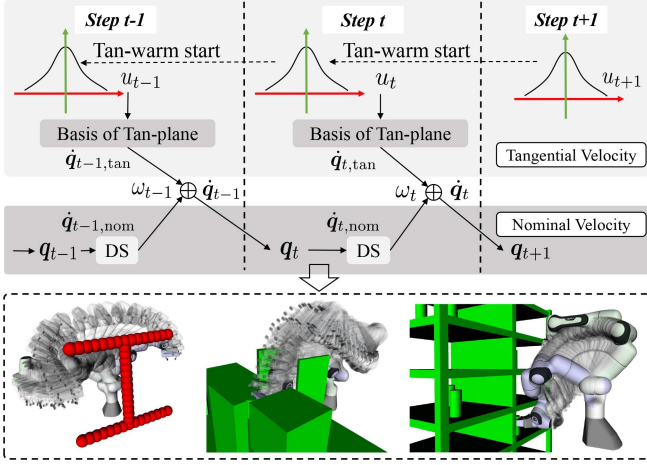


Fig. 1. The architecture of the optimization, where each iteration involves multiple steps (horizons). At each step, we explicitly optimize the tangential velocity. The final velocity is constructed by combining the tangential and nominal velocities with a dynamic weight.

the robot out of local minima. The main contributions are as follows:

- 1) We propose MPPI-Tan to overcome local minima induced by concave obstacles by decomposing the optimal velocity into tangential and nominal components, and explicitly optimizing the tangential velocity along the tangent space basis.
- 2) We introduce a warm start operator in the tangent space and leverage receding horizons to ensure consistency in tangential velocity optimization. Additionally, we adopt a dynamic weight adjustment strategy based on the distance to local minima to form the final velocity.

The rest of this paper is structured as follows. Section II presents the related works, followed by preliminaries in Section III. Whereas Section IV describes the explicit tangent space velocity optimization. The simulation and real experiments are presented in Section V, and the conclusions are drawn in Section VI.

II. RELATED WORKS

Traditional sampling-based methods, such as Rapidly-exploring Random Trees (RRT), Probabilistic Roadmaps (PRM), and their variants [10], [11], [12], are designed to generate collision-free paths. While offering probabilistic completeness, their computational cost can be significant [13]. An alternative class of methods formulates motion planning as an optimization problem, aiming to minimize costs related to collisions and constraints. These include STOMP [14], CHOMP [15], and TrajOpt [16]. However, they are prone to local minima and typically operate offline. Marcucci et al. [17] proposed a compact mixed-integer optimization using Graphs of Convex Sets (GCS), reducing the problem to a convex optimization that enables fast, approximately globally optimal trajectory generation. However, constructing the GCS is time-consuming and must be done offline.

The artificial potential field method [1], a classical approach for online motion generation, repels the robot from obstacles.

However, designing an appropriate potential function is difficult. Alternatives, such as circular fields [18] guide the robot around obstacles, but scaling these fields to high-dimensional spaces remains challenging. The Modulation of Dynamical Systems (ModDS) [2], [3] modulates the original Dynamical Systems (DS) based on the distance to obstacles and the normal direction. However, these methods are typically limited to avoiding collisions with convex and star-shaped obstacles. Additionally, modulation matrices may introduce spurious attractors. Recent work by Fourie et al. [19] introduced on-manifold strategies for navigating non-convex obstacles, using sample-based and projection-based representations for real-time navigation. Haviland et al. [6] reformulated reactive motion control as a quadratic programming problem. However, the approach faces challenges when dealing with local minima. Methods such as [8], [9] apply receding horizons from sampling-based model predictive control [7] to help the robot escape local minima. However, these approaches still rely on specific heuristics and struggle to find feasible solutions in cluttered environments [20], [21].

III. PRELIMINARIES

In this section, we present a concise analysis of the distance function between the robot and its environment, as well as the gradient of the distance function with respect to the joint configuration. Additionally, we provide an illustration of the sampling-based Model Predictive Control (MPC) algorithm.

A. Modeling Environment Collision

We consider a serial robot with m links and n revolute Degrees of Freedom (DOF). The robot's joint state is defined as a vector of joint angles $\mathbf{q} = [q_1, q_2, \dots, q_n]^T \in \mathbb{R}^n$. Since the number of links m typically exceeds the number of DOF n due to fixed joints, we associate the k -th link with the m_k -th actuated joint for convenience. Obstacles $\mathcal{O} \in \mathbb{R}^{d \times 4}$ are represented as either point clouds or a set of spheres, where d denotes the number of points or spheres. The first three columns specify the sphere's center, and the fourth column denotes its radius, which is set to zero for point clouds.

Based on our previous work [22], for a query point $\mathbf{p} \in \mathbb{R}^3$ in the workspace, we utilize a learning-based regression function $NN_{\text{env}}(\mathbf{q}, \mathbf{p})$ to approximate an m -entry vector, where each entry is of the form $[\Gamma_k, \mathbf{n}_k^T]^T \in \mathbb{R}^4$. Here, $\Gamma_k \in \mathbb{R}$ represents the predicted minimal distance between the k -th link and the point $\mathbf{p}$, while $\mathbf{n}_k$ is the predicted unit direction vector from $\mathbf{p}$ to $\mathbf{p}_k$, where $\mathbf{p}_k$ denotes the nearest point on the k -th link. The $\mathbf{p}_k$ is expressed as $\mathbf{p}_k = \mathbf{n}_k \Gamma_k + \mathbf{p}$. The gradient of the k -th distance function $\Gamma_k(\mathbf{q}, \mathbf{p})$ with respect to $\mathbf{q}$ is given by:

$$\nabla_{\mathbf{q}} \Gamma_k = \frac{1}{\Gamma_k(\mathbf{q}, \mathbf{p})} \mathbf{J}_k^T(\mathbf{q})(\mathbf{p}_k - \mathbf{p}) = \mathbf{J}_k^T(\mathbf{q}) \mathbf{n}_k \quad (1)$$

where $\mathbf{J}_k(\mathbf{q}) = \frac{\partial \mathbf{p}_k}{\partial \mathbf{q}} \in \mathbb{R}^{3 \times n}$ is the translation Jacobian matrix of the point $\mathbf{p}_k$ with respect to $\mathbf{q}$. For revolute joints, the i -th column of $\mathbf{J}_k(\mathbf{q})$ is calculated as follows:

$$\mathbf{J}_{k,i}(\mathbf{q}) = \begin{cases} \mathbf{z}_i \times (\mathbf{p}_k - \mathbf{o}_i), & i \leq m_k \\ \mathbf{0}, & m_k < i \leq n \end{cases} \quad (2)$$

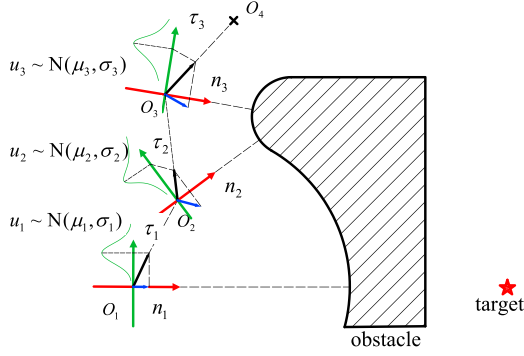


Fig. 2. Motion planning of a mass point, with the negative gradient to obstacles in red, the tangent space basis in green, the nominal velocity in blue, and the final velocity in black. The distribution parameters of control inputs u_i are optimized to compute the tangential velocities, enabling collision-free motion in the presence of a concave obstacle.

where z_i and o_i denote the unit vector along the rotation axis and the position of the i -th joint frame origin, respectively. For brevity, we refer to $\nabla_q \Gamma_{k^*}$, where $k^* = \arg \min_{k \in [1, \dots, m]} \Gamma_k$, as the gradient to obstacles in the following content. Similar to $NN_{\text{env}}(q, p)$, we utilize the implicit self-collision distance function $NN_{\text{self}}(q) : \mathbb{R}^n \rightarrow \mathbb{R}$, adopted from [22], to estimate the minimum distance between the robot's links for a given configuration state q .

B. Sampling-Based MPC

The sampling-based MPC, also known as MPPI, is a stochastic optimization algorithm that aims to find the optimal distribution of control inputs. The key parameters in MPPI are the number of sampled rollouts N and the prediction horizon length H . At the t -th iteration, a set of control inputs $\{u_{i,h}\}_{i=1, \dots, N}^{h=1, \dots, H}$ is sampled from H independent multivariate Gaussian distributions, parameterized by $\pi_{\theta_t} = \{\mu_h, \Sigma_h\}_{h=1, \dots, H}$. The corresponding state rollouts $\{x_{i,h}\}_{i=1, \dots, N}^{h=1, \dots, H}$ are then generated using the approximate dynamics function with the sampled control inputs. The distribution parameters are subsequently updated as follows.

$$\begin{aligned} \mu_{t,h} &= (1 - \alpha_\mu) \mu_{t-1,h} + \alpha_\mu \frac{\sum_{i=1}^N w_i u_{t,h}}{\sum_{i=1}^N w_i} \\ \Sigma_{t,h} &= (1 - \alpha_\sigma) \Sigma_{t-1,h} + \\ &\quad \alpha_\sigma \frac{\sum_{i=1}^N w_i (u_{t,h} - \mu_{t,h})(u_{t,h} - \mu_{t,h})^\top}{\sum_{i=1}^N w_i} \end{aligned} \quad (3)$$

Here, $w_i = \exp\{-C(\{x_{i,h}\}_{h=1, \dots, H})/\beta\}$, where, β is a temperature parameter and $C(\cdot)$ denotes the rollout cost function. α_μ and α_σ represents step-sizes. For more details, refer to [7].

IV. EXPLICIT TANGENT SPACE VELOCITY OPTIMIZATION

The motion planning process for a mass point is illustrated in Fig. 2. The final velocity is decomposed into nominal and tangential components. From this, it is evident that determining the tangential velocity and ensuring its consistency is crucial for

navigating the robot away from local minima. However, the tangent space varies with the joint configuration, making it difficult to represent the tangential velocity in a single space. Therefore, directly using the tangential velocity as an optimization variable poses a challenge. To address this, we first compute the unit orthogonal basis $E(q)$ of the tangent space at a given joint configuration q using the gradient to obstacles $n(q) = \nabla_q \Gamma_{k^*}$ and QR decomposition. The definition of $E(q)$ is as:

$$E(q) = [\tau_1(q), \dots, \tau_{n-1}(q)] \in \mathbb{R}^{n \times (n-1)} \quad (4)$$

where $\tau_i(q)$, for $i = 1, \dots, (n-1)$, form a unit orthonormal basis for the tangent space, with $n(q)$ as its normal. The corresponding tangential velocity is then expressed as $\dot{q}_{\text{tan}} = E(q)u$, where $u = [u_1, u_2, \dots, u_{n-1}]^T \in \mathbb{R}^{n-1}$ represents the components along the tangent space basis. This representation decouples the control input from the varying tangent space, allowing for explicit control of the tangential velocity through the optimization of u . At the t -th iteration, the optimization problem can be formulated as:

$$\underset{U_t \sim \pi_{\theta_t}}{\text{argmin}} \quad C(Q_t, U_t) \quad (5)$$

$$\text{s.t.} \quad \dot{q}_{\text{nom}}(i) = \mathcal{DS}(q(i)) \quad (5.1)$$

$$\dot{q}_{\text{tan}}(i) = E(q(i))u(i) \quad (5.2)$$

$$\dot{q}(i) = \omega_{\text{vel}} \dot{q}_{\text{tan}}(i) + (1 - \omega_{\text{vel}}) \dot{q}_{\text{nom}}(i) \quad (5.3)$$

$$q(i+1) = q(i) + \dot{q}(i)dt \quad (5.4)$$

where, $i = [t, t+1, \dots, t+H-1]$, $\pi_{\theta_t} = \{\mu_{t,h}, \Sigma_{t,h}\}_{h=1, \dots, H}$ represents the parameters of H independent multivariate Gaussian distributions, and dt denotes the control period. The control input over the H -horizon, denoted as $U_t \triangleq [u(t), u(t+1), \dots, u(t+H-1)]$, has each entry sampled from the corresponding Gaussian distribution. The state rollout over the H -horizon, denoted as $Q_t \triangleq [q(t), q(t+1), \dots, q(t+H)]$, has each entry generated using (5.4). The final velocity, as defined in (5.3), is a combination of the tangential velocity $\dot{q}_{\text{tan}}$ and the nominal velocity $\dot{q}_{\text{nom}}$ using a dynamic weight ω_{vel} described in Section IV-C. The nominal velocity, as outlined in (5.1), is derived either from a linear dynamical system $\dot{q}_{\text{nom}} = f(q) = k(q^* - q)/\|q^* - q\|$ or a modulated dynamical system $\dot{q}_{\text{nom}} = M(\cdot)f(q)$, where q^* represents the target configuration, k is a positive amplitude constant and $M(\cdot)$ denotes the modulation matrix, introduced in [2].

We employ the sampled MPC approach to solve the optimization problem (5), and refer to our proposed method as MPPI-Tan. The initial mean μ_0 is set to $\mathbf{0}$, and the initial covariance Σ_0 is set to the identity matrix to promote unbiased exploration in all tangential directions. Since the control inputs are represented in different tangent spaces, to ensure the convergence and consistency of the tangential velocities, we introduce a warm start operator across different tangent spaces, as described in Section IV-B. The flow of the MPPI-Tan algorithm is outlined in Algorithm 1.

Algorithm 1: MPPI-Tan Collision-Free Motion.

Input: Nominal DS: $f(q)$
 Initial and goal joint states: q_{init}, q_f
 Cost function: $C(q, \dot{q})$
 MPPI Parameters: N, H
 Integration interval: Δt
 Joint configurations tolerance: δ

Output: Collision-free motion: $\dot{q}$

Initial current joint states: $q_{\text{cur}} = q_{\text{init}}$

while $\|q_{\text{cur}} - q_f\| > \delta$ **do**
 Initialization: $q_{i,0} = q_{\text{cur}}, \forall i \in [1, N]$
for $i \in [1, N]$ **do**
 for $h \in [1, H]$ **do**
 Update obstacles: $\mathcal{O}$
 Compute gradient to obstacles: $\nabla_{q_{i,h-1}} \Gamma_{k^*}$
 Compute the basis of the tangent plane:
 $E_{i,h-1} = [\tau_1, \tau_2, \dots, \tau_{n-1}]$
 Sample tangential input:
 $\dot{q}_{\text{tan}} = E_{i,h-1} u$, where $u \sim \mathcal{N}(\mu, \Sigma)$
 Evaluate nominal velocity:
 $\dot{q}_{\text{nom}} = f(q_{i,h-1})$
 Compute dynamical weight and synthesis:
 $\dot{q}_{i,h-1} = \omega_{\text{vel}} \dot{q}_{\text{tan}} + (1 - \omega_{\text{vel}}) \dot{q}_{\text{nom}}$
 Integrate: $q_{i,h} = q_{i,h-1} + \dot{q}_{i,h-1} \Delta t$
 Evaluate cost of sampled trajectories:
 $C_i = C(\{q_{i,h}, \dot{q}_{i,h}\}_{h=1 \dots H}), i = 1 \dots N$
 Update parameters of distributions $\mathcal{N}(\mu, \Sigma)$
 Find the optimal velocity:
 $\dot{q}^* = \dot{q}_{i^*,0}$, where $i^* = \arg \min C_i$
 Execute $q_{\text{cur}} = q_{\text{cur}} + \dot{q}^* \Delta t$
 Execute the tangential warm start operator

A. Objective Function

The objective function in the MPPI algorithm defines the expected robot behavior. The objective function can be defined as a weighted sum of several cost terms. In this paper, we consider the following cost terms.

1) *Goal Reaching*: This cost term is designed to guide the robot toward the target joint configuration, q^* , by measuring the distance between the current and target configurations. It is defined as:

$$c_i^{\text{goal}} = \|q^* - q_{i,H}\|_2 \quad (6)$$

where $q_{i,H}$ represents the robot's joint configuration at the end of the i -th rollout. This cost term penalizes rollouts that do not sufficiently approach the target configuration. Notably, only the final configuration of the rollout is considered, as the trajectory may need to temporarily deviate from the goal to escape local minima. This flexibility enables the algorithm to explore a wider range of alternative paths before ultimately converging on an optimal solution.

2) *Consistency Constraint*: This cost term is designed to prevent localized oscillations and back-and-forth movements by penalizing inconsistencies in the tangential velocities during the

initial horizons of the rollouts. It is defined as:

$$c_i^{\text{osc}} = \sum_{t=1}^T \alpha^t \left(1 - \frac{\langle \dot{q}_{i,t}, \dot{q}_{i,t+1} \rangle}{\|\dot{q}_{i,t}\| \|\dot{q}_{i,t+1}\|} \right), \quad t = 1, \dots, T \quad (7)$$

where $\langle \cdot, \cdot \rangle$ represents the dot product, and $\alpha = 0.99$ is a decay factor that assigns greater weight to earlier time horizons while progressively reducing its influence as time progresses. The term $\dot{q}_{i,t}$ denotes the joint velocity at time step t in the i -th rollout, while T , set to 5 in our implementation, represents the number of initial horizons considered. This cost term penalizes unexpected oscillation in the trajectory, which is especially important for ensuring smooth convergence toward the goal.

3) *Path Exploring*: The cost term is designed to prevent the exploration path from becoming excessively short due to the presence of local minima. It is defined as:

$$c_i^{\text{len}} = \frac{\|q^* - q_{i,H}\|_2}{\|q_{i,H} - q_{i,0}\|_2} \quad (8)$$

where $q_{i,0}$ denotes the initial configuration of sampled rollouts. This term encourages exploration by penalizing paths that quickly converge to a local minimum without adequately exploring the configuration space. Additionally, this cost decreases as the robot approaches the target, diminishing its influence when the robot is near the target configuration.

4) *Collision Avoidance*: This cost term is designed to facilitate collision avoidance with both environmental obstacles and self-collisions. We employ a binary collision detection function that penalizes trajectories colliding with the obstacle. The collision avoidance cost is defined as follows:

$$c_i^{\text{coll}} = \sum_{h=1}^H c_{i,h}^{\text{coll}} \quad (9)$$

where $c_{i,h}^{\text{coll}} = \mathcal{I}(\Gamma_{\text{env}}(q_{i,h}) < d_{\text{env}}) + \mathcal{I}(\Gamma_{\text{self}}(q_{i,h}) < d_{\text{self}})$. Here, $\Gamma_{\text{env}}(q)$ and $\Gamma_{\text{self}}(q)$ denote the environment distance and self-distance estimators, respectively. The function $\mathcal{I}(\cdot)$ represents the indicator function, while d_{env} and d_{self} correspond to the safe distances for the environment and self-collision, respectively.

B. Warm Start Operator

Different joint configurations correspond to distinct tangent spaces, as shown in Fig. 3(a). For the given joint configurations q_1 and q_2 , the corresponding basis of the tangent spaces are denoted as $E(q_1) = [\tau_{11}, \tau_{12}]$ and $E(q_2) = [\tau_{21}, \tau_{22}]$, respectively. As described above, the optimal distribution parameters are presented on the basis of the tangent spaces. To ensure convergence and maintain consistency in the tangential velocity, we introduce a warm start operator, $\Phi(\theta)$, for optimizing tangential velocity. This operator transforms the previous approximate solution θ_{t-1} as the initialization for the current iteration, resulting in $\hat{\theta}_t = \Phi(\theta_{t-1})$, as illustrated in Fig. 3(b). When the distribution parameters are transformed from q_1 to q_2 , the warm start operator $\Phi(\theta)$ is defined as follows:

$$\hat{\mu}_2 = E(q_2)^\dagger E(q_1) \mu_1$$

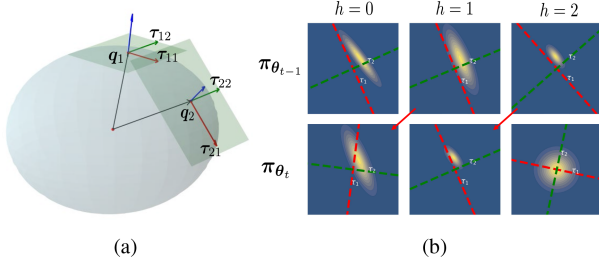


Fig. 3. The warm start operator. (a) Tangential hyperplanes and their basis at joint configurations q_1 and q_2 for a 3D obstacle. (b) The control distribution π_{θ} consists of $H = 3$ independent Gaussian distributions, where the warm start operator moves the distribution parameters one time step forward and replaces those at $h = H - 1$ with default values.

$$\hat{\Sigma}_2 = E(q_2)^\dagger E(q_1) \Sigma_1 E(q_1)^T E(q_2)^\dagger \quad (10)$$

where, $E(q_2)^\dagger$ denotes the pseudo-inverse of the matrix $E(q_2)$, $\hat{\mu}_2$ and $\hat{\Sigma}_2$ represent the initialization mean and covariance matrix at q_2 , respectively.

C. Dynamic Velocity Weight

As the robot approaches a local minimum, the influence of the tangential velocity increases, while it decreases when the robot moves away from it. We quantify the proximity to local minima using the dot product between the gradient to obstacle and the nominal velocity direction, $\langle n(q), \dot{q}_{nom} \rangle$. The final velocity is defined as:

$$\dot{q} = \omega_{vel} \dot{q}_{tan} + (1 - \omega_{vel}) \dot{q}_{nom} \quad (11)$$

where $\omega_{vel}(q) = \omega_\Gamma(q) \omega_n(q) \mathcal{I}(\|q - q^*\| > \delta)$, $\mathcal{I}(\cdot)$ denotes the indicator function. The indicator function ensures the asymptotic stability properties of the nominal DS by enforcing $\omega_{vel}(q) = 0$ within a ball of radius δ around the target configuration q^* , assuming no obstacles exist within this region. We define $\omega_\Gamma(q)$ and $\omega_n(q)$ using parameterized functions, allowing the hyperparameters to be adjusted to improve the probability of finding effective trajectories. The definitions of $\omega_\Gamma(q)$ and $\omega_n(q)$ are as follows:

$$\begin{aligned} \omega_n(q) &= 1 - \frac{1}{1 + e^{-k_n \langle n(q), \dot{q}_{nom} \rangle}} \\ \omega_\Gamma(q) &= 1 - \frac{1}{1 + e^{-k_\Gamma (\Gamma(q) - d_s)}} \end{aligned} \quad (12)$$

where $\Gamma(q)$ represents the distance to the nearest obstacle, d_s denotes the safe distance threshold. The positive parameters k_n and k_Γ control the transition widths. Fig. 4(a) shows the curve of $\omega_n(q)$. A larger k_n makes $\omega_n(q)$ more sensitive to local minima, enhancing the role of tangential velocity. A smaller k_n facilitates exploration near local minima. A larger k_Γ causes ω_Γ to approach 1 only near the safety distance d_s , making the trajectory stays close to obstacles.

The dynamic weight helps mitigate obstacle penetration in continuous cases. As defined in [2], impenetrability is ensured when the velocity field's projection onto the obstacle gradient, $n(q)$, vanishes on the obstacle surface $\partial\mathcal{O}$, mathematically formulated as $n(q)^T \dot{q} = 0$ for all $q \in \partial\mathcal{O}$. With the

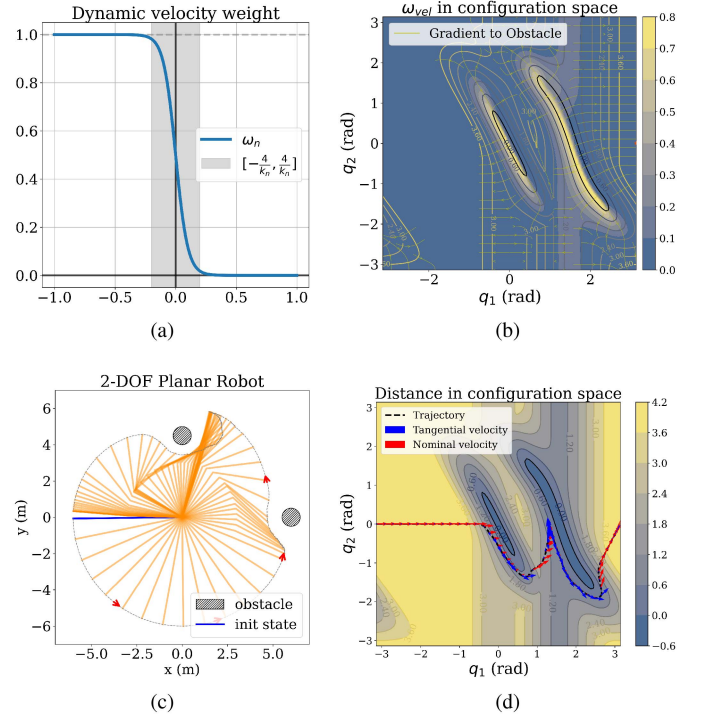


Fig. 4. Online motion generation of a 2-DoF planar serial robot using MPPI-Tan. (a) The curve of ω_n . (b) The contour plot of the dynamic weight ω_{vel} in configuration space. (c) Motion from $(-\pi, 0)$ to $(\pi, 0)$ in task space. (d) Motion in the configuration space, with tangential and nominal velocities marked in blue and red, respectively.

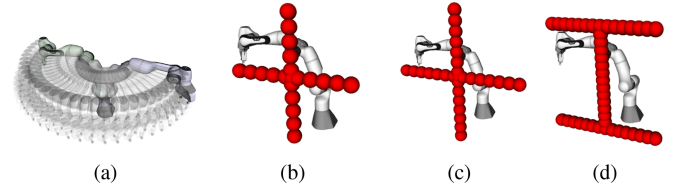


Fig. 5. The reaching task for the Franka robot. (a) The default joint motion to drive the robot from the initial position (in light-green) to the goal position (in light-purple) in the reaching task. (b), (c) and (d) represent the cross-shaped obstacles with sizes of 5 and 7, as well as an I-shaped obstacle, placed in the robot's swept area.

velocity defined as (11), the projection becomes $n(q)^T \dot{q} = (1 - \omega_{vel}) n(q)^T \dot{q}_{nom}$. When $n(q)^T \dot{q}_{nom} < 0$ or the distance to obstacles approaches the safe threshold, the dynamic weight ω_{vel} tends to 1, which helps mitigate obstacle penetration.

V. SIMULATIONS AND PHYSICAL EXPERIMENTS

We evaluated the MPPI-Tan algorithm through simulations and physical experiments. First, we deployed it on a 2-DOF planar serial robot to analyze the role of tangential velocity in overcoming local minima. We then applied the algorithm to a 7-DOF Franka robot in simulations for a reaching task and the Motion benchmark. Finally, we validated the algorithm through physical experiments. The implementation was in Python, running on an AMD Ryzen R4600H 2.80 GHz CPU with 16 GB RAM and an RTX1650 GPU. In our implementation, we set the number of sampled rollouts N to 50, the prediction horizon

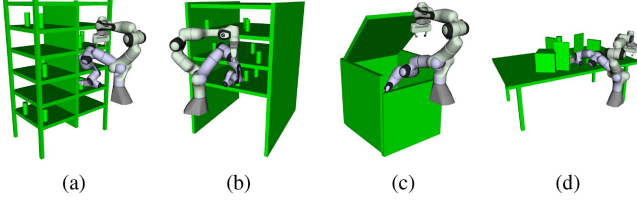


Fig. 6. The selected scenes used to test the performance from MotionBenchmaker. The initial joint state is in light-green and the goal joint state is in light-purple. (a) Thin Bookshelf (b) Tall Bookshelf (c) Box (d) Table Pick.

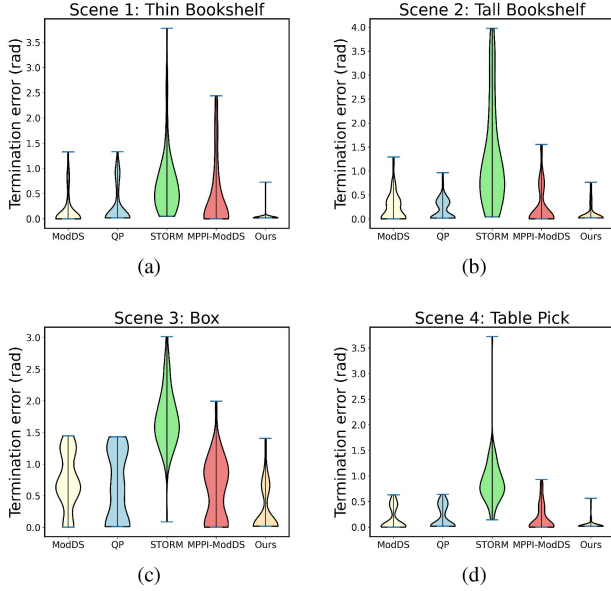


Fig. 7. Violin plot of termination error across different scenarios for each algorithm. The termination error represents the 2-norm distance between the final joint configuration and the target configuration.

length H to 15, and the integration time step to 0.2 seconds. For all baseline algorithms, we used their default hyperparameters and maintained consistent safety distance thresholds: 0.1m for the reaching task and 0.05 m for the Motion benchmark.

In each experiment, the robot moved from an initial joint configuration to a target configuration, with the success rate as the performance metric. A test was considered successful if the final joint configuration was within a 2-norm distance of 0.02 radians from the target.

A. 2-DOF Planar Serial Robot

To illustrate the effect of the MPPI-Tan method, we deployed it on a 2-DOF planar serial robot with link lengths $l_1 = l_2 = 3$. The initial joint configuration was set to $(-\pi, 0)$, and the target was $(\pi, 0)$. Two circular obstacles, each with a radius of 0.5, were placed at $(6, 0)$ and $(0, 4.5)$.

The generated motion is illustrated in Fig. 4(c), and its motion in configuration space is depicted in Fig. 4(d), with tangential and nominal velocities marked in blue and red, respectively. The nominal velocities are derived from a linear dynamical system. From the motion in Fig. 4(d), we observe that the dynamic weight reduces the nominal velocity and enhances the tangential velocity near local minima. The contour plot of the dynamic weight ω_{vel} in configuration space is shown in Fig. 4(b),

TABLE I
PERFORMANCE COMPARISON OF REACHING TASK

Scene Name	ModDS	Success Rate (%)		
		QP	MPPI-ModDS	MPPI-Tan
Cross-shaped 5	62	66	100	100
Cross-shaped 7	51	53	100	99
I-shaped	—	—	93	90

The bold values indicate the highest success rates among the compared methods for each test task.

where the weight is higher around local minima and the safety distance d_s , ensuring that the tangential velocity dominate in these regions. This allows the robot to escape local minima and avoid collisions with obstacles effectively. The experimental results demonstrate that the incorporation of tangential velocity significantly enhances the robot's capability to escape local minima. Moreover, the MPPI-Tan algorithm effectively mitigates oscillations in tangential velocities, ensuring their consistency and stability during motion.

B. 7-DoF Franka Robot–Reaching Task

To facilitate comparison with other algorithms, we consider the reaching task from [5], where the robot performs a reaching motion towards a goal configuration while avoiding obstacles. The task setup is shown in Fig. 5(a), with static cross-shaped (Fig. 5(b) and (c)) or I-shaped (Fig. 5(d)) obstacles placed in front of the robot, segmenting its workspace into quarters. For each obstacle, we conducted 100 repeated reaching motions, with slight random variations in the obstacle positions. The original centers of the cross-shaped and I-shaped obstacles are located at $(0.4, 0, 0.5)$ and $(0.4, 0, 0.425)$ relative to the robot's base frame.

We compare our method with ModDS [2], the QP-based collision-free motion generation algorithm [22] (referred to simply as QP), and the MPPI-based Modulated Dynamical System (MPPI-ModDS) [5]. The results are presented in Table I, where “—” indicates failure to accomplish the task.

C. 7-DoF Franka Robot–Motion Benchmark

To comprehensively evaluate the performance of our proposed method, we selected 400 problems across four different test scenes and used them as motion benchmarks. The test scenes are part of a predefined selection in the MotionBenchmaker [23] for the Panda robot. The selected test scenes are Thin Bookshelf, Tall Bookshelf, Box, and Table Pick, as shown in Fig. 6. In each problem, obstacle poses are defined relative to the robot's base coordinate frame, with specified initial and target joint configurations. The motion planners are tasked with online generating a trajectory between the initial and target configurations while ensuring collision avoidance. We used spherical approximations to represent these scenes, with the number of the spheres ranging from 300 to 1000.

We added comparison experiments with STORM [8], which uses the MPPI algorithm to directly optimize joint accelerations or velocities for collision-free motion generation. We modified the original goal cost to operate in joint space. Algorithms are terminated either when the target configuration is reached or

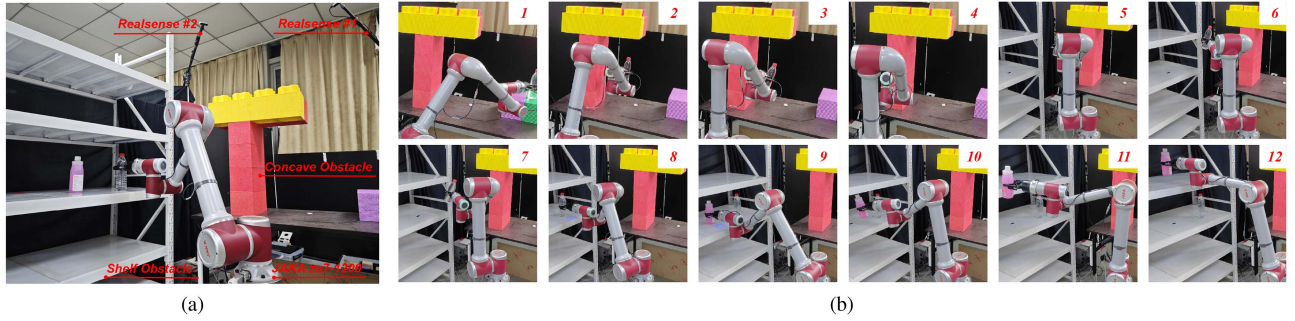


Fig. 8. The Real robot experiment (a) The experimental setup. (b) Snapshots from a collision-free motion planning experiment of MPPI-Tan, where the robot moved to transport and repack bottles between concave obstacles and a shelf. Sequences (1–4) illustrate the robot transferring a bottle through concave obstacles, sequences (5–8) show the robot maneuvering around the sides of the shelf, and sequences (9–12) depict the robot retrieving a bottle from the shelf and rearranging it across different levels within the shelf.

TABLE II
PERFORMANCE COMPARISON OF BENCHMARK

Scene Name	ModDS	Success Rate (%)		
		QP	MPPI-ModDS	MPPI-Tan
Thin Bookshelf	68	63	56	97
Tall Bookshelf	50	55	67	81
Box	23	40	29	65
Table Pick	56	54	58	80

The bold values indicate the highest success rates among the compared methods for each test task.

TABLE III
CALCULATION FREQUENCY COMPARISON

Method	ModDS	QP	STORM	MPPI-ModDS ($H = 10$)
Freq. (Hz)	472.1	149.3	84.2	408.3 (7.2)
Method	MPPI-Tan			
	$H = 2$	$H = 5$	$H = 10$	$H = 15$
Freq. (Hz)	95.3	41.3	20.9	14.2

when the maximum iteration limit, determined by the planning time limit, is exceeded. The termination error is defined as the 2-norm distance between the final and target configurations. The termination error for different algorithms is shown in Fig. 7, and the success rates are presented in Table II. During our experiments, the termination error of the STORM method failed to meet the tolerance threshold for the 2-norm distance to the target configuration, which resulted in a low success rate within our set tolerance. Therefore, we excluded its success rate from Table II.

D. Simulation Results and Complexity Analysis

From the simulation results, MPPI-Tan outperforms state-of-the-art algorithms in both success rate and termination error across the tested scenarios. Compared to STORM, MPPI-Tan benefits from explicitly optimizing tangential velocity rather than directly optimizing joint velocities. The latter makes it challenging to escape local minima. In contrast, MPPI-Tan reduces the optimization space's dimensionality by optimizing tangential velocity in the tangent space, introducing a heuristic search that simplifies the optimization in high-dimensional, non-convex spaces and improves its ability to avoid local minima. Additionally, occasional collisions were observed in motions generated by STORM, as it relies solely on a cost function to penalize collisions, without guaranteeing complete obstacle avoidance. MPPI-ModDS optimizes the modulation of Gaussian kernels within the configuration space, refining the potential function of the original dynamic system. In the reaching task, MPPI-ModDS performs slightly better than MPPI-Tan, where MPPI-ModDS typically requires optimizing only 2-3 Gaussian

kernels, making it easier to optimize the deflection direction. However, MPPI-ModDS performs worse in cluttered scenarios compared to sparser ones, like the reaching task. In contrast, MPPI-Tan remains effective in both cluttered and sparse environments, demonstrating its robustness.

In addition to the success rate, we compare the mean computation frequency of different algorithms across varying obstacle sizes, as shown in Table III, where H denotes the number of predicted horizons. Since MPPI-ModDS operates asynchronously, the two frequencies correspond to the DS frequency and the obstacle-tangent deflection optimization frequency. The computational complexity of our algorithm scales linearly with the prediction horizon H . STORM achieves high-frequency motion planning by using a linear approximation of the robot's forward kinematics, enabling parallelized computation of the rollout integration within a sampling-based MPC framework. In contrast, our algorithm introduces tangential velocity at each timestep, making the system nonlinear and increasing the difficulty of parallelizing the multi-horizon optimization. The results in Table III indicate that H significantly impacts the computation frequency. Therefore, a dynamic prediction horizon adjustment strategy, based on the distance from local minima, could improve the efficiency of MPPI-Tan.

To provide a detailed breakdown, we further analyzed the runtime of the main computational components. The neural network $NN_{\text{env}}(\mathbf{q}, \mathbf{p})$, which computes the nearest distances and gradients between the robot and obstacles, accounts for approximately 70% of the total computation time. Its computational complexity is $O(mB(n_oN_l + DN_l^2))$, where n_o is the input dimension (3 in our case), m is the number of robot links, B is the batch size, N_l is the number of neurons per layer, and D is the network depth. In our implementation, B can be parallelized, but the distance

inference for m robot links is performed sequentially. Therefore, parallelizing the network computations across all links would significantly improve computational efficiency.

E. Real Robot Experiments

We further validated the performance of MPPI-Tan on a JAKA Zu7-1200 robot with 6-DOF. The experimental setup, shown in Fig. 8(a), features a scene containing concave obstacles and a shelf. The experiment involves moving the robot to transport and repack bottles between the concave obstacles and the shelf. The scenario primarily encompasses three stages: 1) The robot navigates to avoid concave obstacles along its motion path. 2) The robot avoids the sides of the shelf and places the bottle onto the shelf. 3) The robot retrieves the bottle from the shelf and rearranges it across different levels within the shelf.

Stage 1 primarily aims to test the algorithm's capability to navigate around concave obstacles, while stages 2 and 3 assess its performance in narrow scenarios. Fig. 8(b) showcases snapshots of a successful motion where the robot avoids colliding with concave obstacles. The supplementary video provides a comprehensive view of the entire experiment process. The robot successfully prevents collisions, navigates out of concave areas, and completes its tasks. The physical experiment results indicate that several iterations are required to determine the appropriate tangential velocities for the robot to escape local minima in narrow scenarios. This is mainly because the MPPI algorithm relies on continuous sampling to solve the optimal distribution parameters. Addressing this challenge presents a promising direction for future research.

VI. CONCLUSION

In this paper, we proposed MPPI-Tan, an approach for online collision-free motion generation in configuration space. Our approach decouples the tangential velocity from the varying tangent space and explicitly optimizes tangential velocity components along the tangent space basis, enabling the robot to escape from local minima. Additionally, we introduce a warm-start operator in the tangent space to ensure both the convergence and consistency of tangential velocities. The optimal velocity is constructed by combining the tangential and nominal velocities, with a dynamic weight that adjusts based on the proximity to local minima. MPPI-Tan demonstrated superior performance over state-of-the-art algorithms in generating collision-free motions, particularly in complex and cluttered environments. However, the algorithm's computational complexity currently restricts its application to static and quasi-static scenarios. Improving computational efficiency is a key focus for future work to extend the applicability of our approach to more environments.

REFERENCES

- [1] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *Int. J. Robot. Res.*, vol. 5, no. 1, pp. 90–98, 1986.
- [2] S. M. Khansari-Zadeh and A. Billard, "A dynamical system approach to realtime obstacle avoidance," *Auton. Robots*, vol. 32, pp. 433–454, 2012.
- [3] L. Huber, A. Billard, and J.-J. Slotine, "Avoidance of convex and concave obstacles with convergence ensured through contraction," *IEEE Robot. Automat. Lett.*, vol. 4, no. 2, pp. 1462–1469, Apr. 2019.
- [4] L. Huber, J.-J. Slotine, and A. Billard, "Avoiding dense and dynamic obstacles in enclosed spaces: Application to moving in crowds," *IEEE Trans. Robot.*, vol. 38, no. 5, pp. 3113–3132, Oct. 2022.
- [5] M. Koptev, N. Figueroa, and A. Billard, "Reactive collision-free motion generation in joint space via dynamical systems and sampling-based MPC," *Int. J. Robot. Res.*, vol. 43, no. 13, pp. 2049–2069, 2024.
- [6] J. Haviland and P. Corke, "NEO: A novel expeditious optimisation algorithm for reactive motion control of manipulators," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 1043–1050, Apr. 2021.
- [7] G. Williams, A. Aldrich, and E. A. Theodorou, "Model predictive path integral control: From theory to parallel computation," *J. Guid., Control, Dyn.*, vol. 40, no. 2, pp. 344–357, 2017.
- [8] M. Bhardwaj et al., "STORM: An integrated framework for fast joint-space model-predictive control for reactive manipulation," in *Proc. 5th Conf. Robot Learn.*, 2022, pp. 750–759.
- [9] C. Pezzato, C. Salmi, E. Trevisan, M. Spahn, J. Alonso-Mora, and C. Hernández Corbato, "Sampling-based model predictive control leveraging parallelizable physics simulations," *IEEE Robot. Automat. Lett.*, vol. 10, no. 3, pp. 2750–2757, Mar. 2025.
- [10] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *IEEE Trans. Robot. Automat.*, vol. 12, no. 4, pp. 566–580, Aug. 1996.
- [11] J. J. Kuffner and S. M. LaValle, "RRT-connect: An efficient approach to single-query path planning," in *Proc. ICRA. Millennium Conf. IEEE Int. Conf. Robot. Automat. Symposia Proc.*, 2000, pp. 995–1001.
- [12] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Batch informed trees (BIT*): Sampling-based optimal planning via the heuristically guided search of implicit random geometric graphs," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2015, pp. 3067–3074.
- [13] Z. Kingston, M. Moll, and L. E. Kavraki, "Sampling-based methods for motion planning with constraints," *Annu. Rev. Control, Robot., Auton. Syst.*, vol. 1, pp. 159–185, 2018.
- [14] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, "STOMP: Stochastic trajectory optimization for motion planning," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2011, pp. 4569–4574.
- [15] M. Zucker et al., "CHOMP: Covariant hamiltonian optimization for motion planning," *Int. J. Robot. Res.*, vol. 32, no. 9/10, pp. 1164–1193, 2013.
- [16] J. Schulman et al., "Motion planning with sequential convex optimization and convex collision checking," *Int. J. Robot. Res.*, vol. 33, no. 9, pp. 1251–1270, 2014.
- [17] T. Marcucci, M. Petersen, D. von Wrangel, and R. Tedrake, "Motion planning around obstacles with convex optimization," *Sci. Robot.*, vol. 8, no. 84, 2023, Art. no. eadf7843.
- [18] M. Becker, T. Lilge, M. A. Müller, and S. Haddadin, "Circular fields and predictive multi-agents for online global trajectory planning," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 2618–2625, Apr. 2021.
- [19] C. K. Fourie, N. Figueroa, and J. A. Shah, "On-manifold strategies for reactive dynamical system modulation with nonconvex obstacles," *IEEE Trans. Robot.*, vol. 40, pp. 2390–2409, 2024.
- [20] J. Sacks and B. Boots, "Learning to optimize in model predictive control," in *Proc. Int. Conf. Robot. Automat.*, 2022, pp. 10549–10556.
- [21] H. -J. Yoon, C. Tao, H. Kim, N. Hovakimyan, and P. Voulgaris, "Sampling complexity of path integral methods for trajectory optimization," in *Proc. Amer. Control Conf.*, 2022, pp. 3482–3487.
- [22] G. Zhao, J. Wu, and Z. Xiong, "Perceptual local collision-free motion generation for manipulators," *IEEE Robot. Automat. Lett.*, vol. 10, no. 1, pp. 208–215, Jan. 2025.
- [23] C. Chamzas et al., "MotionBenchMaker: A tool to generate and benchmark motion planning datasets," *IEEE Robot. Automat. Lett.*, vol. 7, no. 2, pp. 882–889, Apr. 2022.